

Practical Lab Report

Part 2: SQL Injection in a Controlled Environment (Kali Linux + DVWA)

Field	Details
Student Name	Joseph Osemudiamhen Okoekoh
Student ID	CSC/22/01/0047
Course	COMP404 Computer Security
Lecturer	Dr. George K. Agordzo
Lab Environment	Kali Linux (192.168.239.128) attacking DVWA on Metasploitable2-Linux (192.168.239.129)
Date	23 May 2026

1. Introduction

This report documents a controlled SQL Injection exercise carried out against the SQL Injection module of the Damn Vulnerable Web Application (DVWA), hosted on the Metasploitable2 target (192.168.239.129) and accessed from the Kali Linux workstation (192.168.239.128). The objective was to understand how SQL Injection vulnerabilities arise, demonstrate a manual authentication-bypass style injection, observe the underlying HTTP traffic using Burp Suite, and attempt automated detection and exploitation using sqlmap. DVWA's security level was set to Low for this exercise, as required by the lab instructions, and all testing was confined to the authorized lab target.

2. Part A: Manual SQL Injection (Understanding the Concept)

The DVWA SQL Injection page was opened at:

```
http://192.168.239.129/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
```

With the DVWA security level set to Low, a basic test was first performed by entering 1 in the User ID field. The application returned a single matching record:

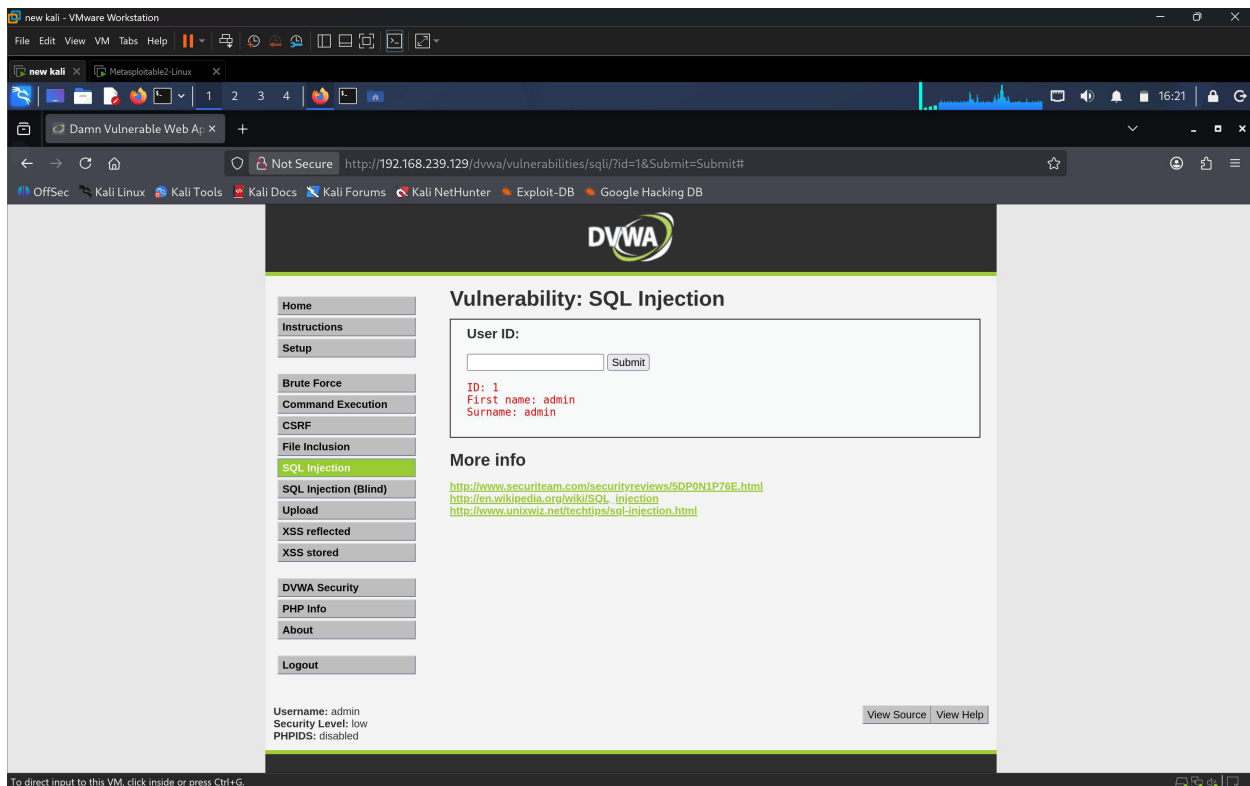


Figure 1: Normal input (User ID = 1) returns a single user record (ID: 1, admin / admin).

Next, the SQL Injection payload `1' OR '1'='1` was entered into the User ID field. The browser sent this as the query string `id=1'+OR+'1'%3D'1&Submit=Submit`. Instead of returning a single user, the application returned the records for every user in the table - admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith - demonstrating a classic authentication-bypass style SQL Injection:

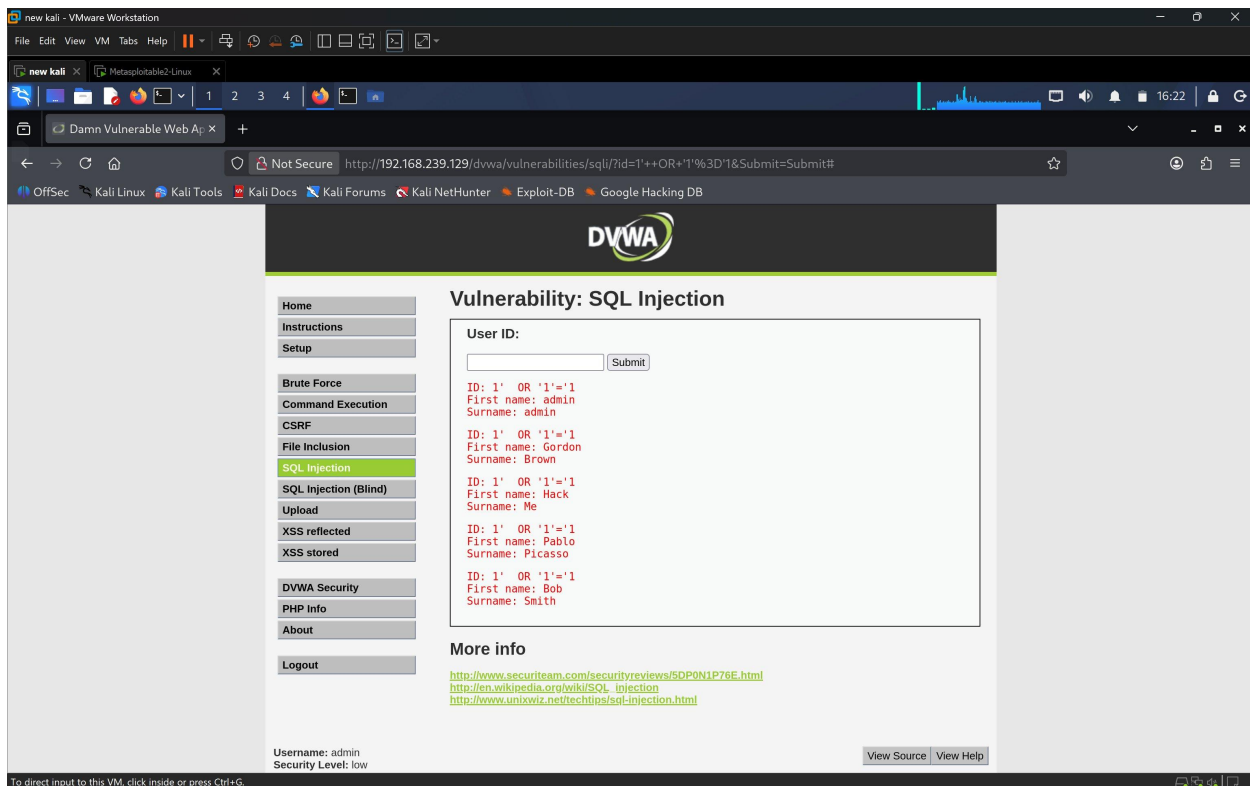


Figure 2: Injected input (`1' OR '1'='1'`) returns all five user records, confirming SQL Injection.

2.1 Explanation

The underlying query executed by the application is most likely structured as:

```
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';
```

Because the OR `'1'='1'` condition is always true, the WHERE clause matches every row in the users table regardless of the user_id value supplied. The application then loops over all matching rows and displays them, so an attacker who only intended to look up a single record receives the entire table instead. In a login form, the equivalent payload could be used to bypass authentication entirely, since the WHERE clause used to check a username/password pair would also evaluate to true for every row.

3. Part B: Using Burp Suite (Traffic Analysis)

Burp Suite Community Edition was started with Intercept turned on, and the browser proxy was configured to route traffic through Burp on 127.0.0.1:8080. A request to the SQL Injection page was then intercepted:

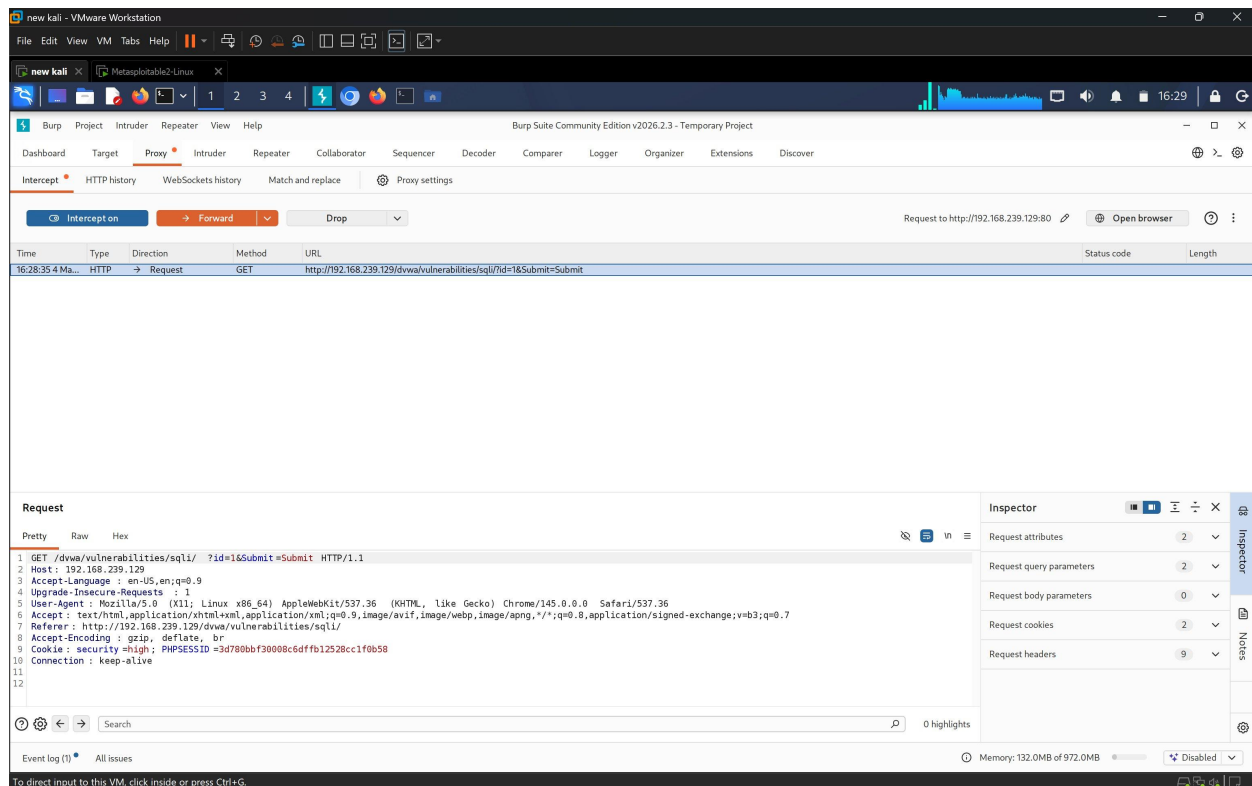


Figure 3: Burp Suite intercepting a GET request to `/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit`.

The intercepted request shows a GET request to `/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit` sent over plain HTTP to 192.168.239.129. The `id` parameter is passed directly in the URL query string with no visible encoding, validation, or sanitization applied by the client. The Cookie header also shows the session identifiers `PHPSESSID` and the DVWA security level cookie (`security`), both transmitted in clear text, confirming that the `id` value supplied by the user is passed straight through to the server-side query exactly as typed - which is what allows the `1' OR '1'=1` payload from Part A to alter the SQL query's logic.

4. Part C: Using sqlmap (Automated Testing - Lab Only)

The same vulnerable URL was passed to sqlmap in non-interactive mode:

```

sqlmap -u
"http://192.168.239.129/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit"
--batch

```

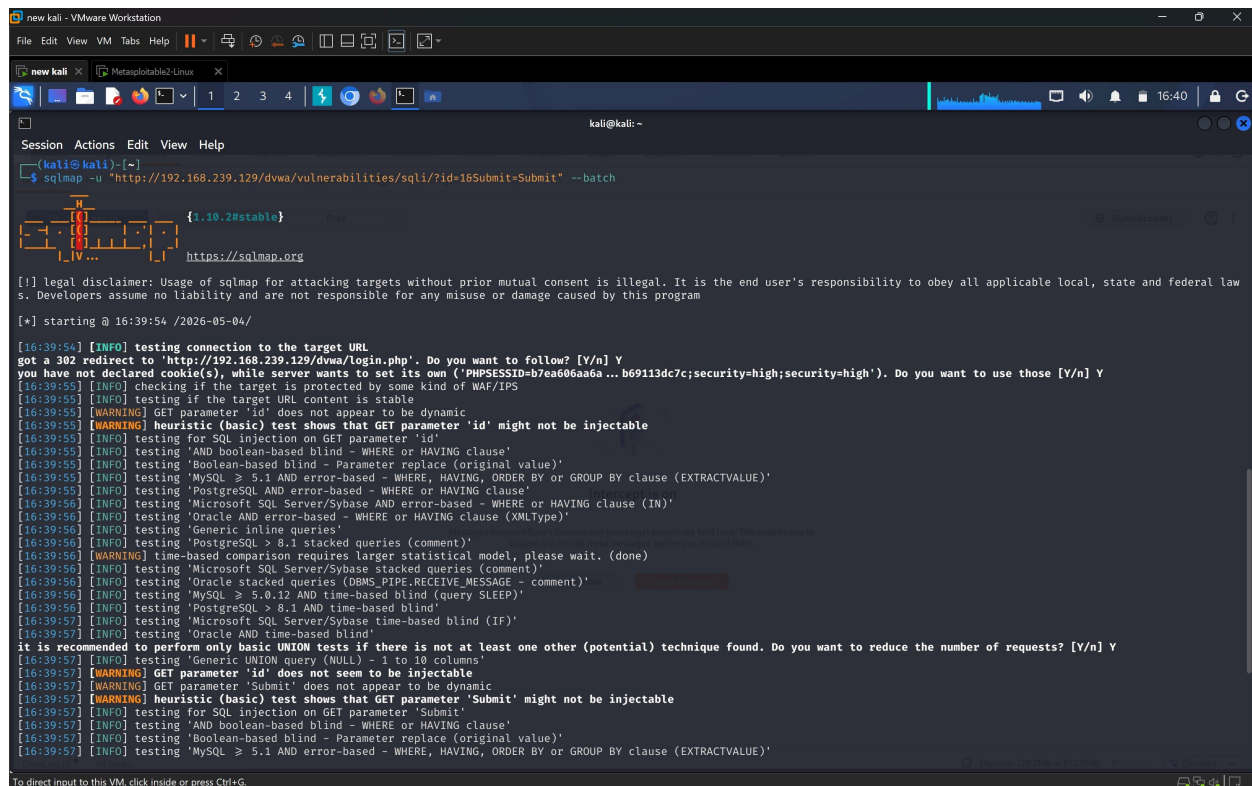


Figure 4: *sqlmap* (--batch) testing the *id* and *Submit* GET parameters for SQL injection.

sqlmap (v1.10.2) followed the 302 redirect to the DVWA login page, accepted the server's session cookies (PHPSESSID and security=high), and then ran through its standard battery of tests - boolean-based blind, error-based (MySQL, PostgreSQL, MSSQL, Oracle), stacked queries, time-based blind, and UNION-based techniques - against both the id and Submit GET parameters. In this run, sqlmap's heuristic and injection tests reported that the id parameter "does not seem to be injectable" via this request.

A second scan was run to attempt database enumeration:

```
sqlmap -u  
"http://192.168.239.129/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit"  
--dbs
```

Figure 5: sqlmap (--dbs) repeating the injection tests in preparation for database enumeration.

4.1 Discussion of sqlmap Results

Although Part A clearly demonstrated a working SQL Injection through the browser, sqlmap's automated run against the plain `id=1&Submit=Submit` request did not classify the `id` parameter as injectable, and the `--dbs` run followed the same pattern. This is a useful real-world observation: DVWA's SQL Injection page requires an authenticated session with the security level cookie set, and the exact cookie values sqlmap auto-accepted (`security=high`, a fresh `PHPSESSID`) differed from the session and `security=low` context used for the successful manual test in Part A. Because sqlmap was testing a different session/security context than the one used manually, its heuristic tests on the raw `id` value did not trigger the same vulnerable code path.

This does not mean the application is secure - Part A and Part B already proved that unsanitized input in the `id` parameter changes the SQL query's behaviour. It does illustrate that automated scanners can produce false negatives when session state, cookies, or application security settings are not aligned with the vulnerable code path, and that manual testing (as in Part A) remains an essential complement to automated tools such as sqlmap.

5. Explanation of the Vulnerability

The DVWA SQL Injection page concatenates the user-supplied `id` value directly into a SQL `SELECT` statement without any input validation, parameterization, or escaping. As demonstrated in Part A, an attacker can alter the structure and logic of the underlying query by supplying SQL syntax (e.g. `' OR '1'='1`) instead of the expected numeric ID. This allows an attacker to retrieve data they are not authorized to see (in this case, every user record instead of one), and in more

complex scenarios could be extended to read other tables, modify data, or - depending on database privileges - execute administrative operations.

6. Risk Level

Risk Level: High

This is rated High because the vulnerability allows complete bypass of the intended access control on the SQL Injection page with a trivial, well-known payload, exposing the full contents of the users table (including names associated with admin accounts). Combined with the lack of HTTPS observed in Part B (which exposes session cookies in transit), an attacker on the same network could both inject malicious input and hijack an authenticated session, significantly amplifying the impact.

7. Recommendations / Prevention Techniques

7.1 Input Validation and Parameterized Queries

The most effective fix is to never concatenate user input directly into SQL statements. Instead, use parameterized queries (prepared statements), which separate the SQL code from the data supplied by the user:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");  
$stmt->execute([$id]);
```

With this approach, the value of \$id is always treated as data, never as part of the SQL command, so payloads such as 1' OR '1'=1 cannot alter the query's logic.

7.2 Other Controls

- Use an ORM (Object-Relational Mapping) framework, which generates parameterized queries by default and reduces the chance of developers writing raw, concatenated SQL.
- Apply strict input validation/sanitization on the id parameter - for example, reject any value that is not a positive integer before it reaches the database layer.
- Disable detailed error messages in production so that database error text (which can reveal table/column names) is never returned to the client.
- Deploy a Web Application Firewall (WAF) in front of the application to detect and block common SQL Injection patterns as a defense-in-depth measure, while the underlying code is fixed.
- Enforce least-privilege database accounts for the web application, so that even if injection occurs, the impact is limited to what that account is permitted to read or modify.
- Serve the application over HTTPS and apply Secure/HttpOnly flags to session cookies, reducing the risk of session hijacking alongside injection-based attacks.

8. Conclusion

This exercise demonstrated a complete, classic SQL Injection workflow against DVWA: a manual payload (1' OR '1'=1) successfully bypassed the intended query logic and returned every user record from the database; Burp Suite confirmed that the vulnerable id parameter is sent unmodified to the server over an unencrypted HTTP connection; and sqlmap, while a powerful

automated tool, did not flag the same parameter as injectable under the session/security context it auto-negotiated, highlighting the continued importance of manual verification. The vulnerability is assessed as High risk, and the recommended remediation - principally the use of parameterized queries / prepared statements, combined with input validation, least-privilege database accounts, and HTTPS - would close this vulnerability and significantly reduce the application's overall attack surface.